

zadania

Zadanie 1. Napisz funkcję zwracającą wyznacznik macierzy kwadratowej dowolnego rozmiaru.

```
import math

def minor(macierz, usun_wiersz, usun_kolumne):
    wynik = []

    for i in range(len(macierz)):
        if i == usun_wiersz:
            continue

        nowy_wiersz = []
        for j in range(len(macierz[i])):
            if j == usun_kolumne:
                continue
            nowy_wiersz.append(macierz[i][j])

        wynik.append(nowy_wiersz)

    return wynik

def wyznacznik_macierzy(macierz):
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie jest kwadratowa")

    if n == 1:
        return macierz[0][0]

    if n == 2:
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0]

    det = 0
    for j in range(n):
        podmacierz = minor(macierz, 0, j)
        det += math.pow((-1), 0+j) * macierz[0][j] * wyznacznik_macierzy(podmacierz)

    return det

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

print("Wyznacznik macierzy: ", wyznacznik_macierzy(macierz))
```

Zadanie 2. Napisz funkcję zwracającą transpozycję macierzy dowolnego rozmiaru.

PYTHON

```
def transpozycja(macierz):
    liczba_wierszy = len(macierz)
    liczba_kolumn = len(macierz[0])

    wynik = []

    for j in range(liczba_kolumn):
        nowy_wiersz = []
        for i in range(liczba_wierszy):
            nowy_wiersz.append(macierz[i][j])
        wynik.append(nowy_wiersz)

    return wynik

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

print("Przed transpozycją macierzy:")
for wiersz in macierz:
    print(wiersz)

wynik = transpozycja(macierz)

print("Po transpozycji macierzy:")
for wiersz in wynik:
    print(wiersz)
```

Zadanie 3. Napisz funkcję znajdującą macierz odwrotną do macierzy kwadratowej dowolnego rozmiaru za pomocą:

- a) rozwinięcia Laplace'a,
- b) metody Gaussa-Jordana.

```
import math

def minor(macierz, usun_wiersz, usun_kolumne):
    wynik = []

    for i in range(len(macierz)):
        if i == usun_wiersz:
            continue

        nowy_wiersz = []
        for j in range(len(macierz[i])):
            if j == usun_kolumne:
                continue
            nowy_wiersz.append(macierz[i][j])

        wynik.append(nowy_wiersz)

    return wynik

def wyznacznik_macierzy(macierz):
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie jest kwadratowa")

    if n == 1:
        return macierz[0][0]

    if n == 2:
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0]

    det = 0
    for j in range(n):
        podmacierz = minor(macierz, 0, j)
        det += math.pow((-1), 0+j) * macierz[0][j] * wyznacznik_macierzy(podmacierz)

    return det

def transpozycja(macierz):
    liczba_wierszy = len(macierz)
    liczba_kolumn = len(macierz[0])

    wynik = []

    for j in range(liczba_kolumn):
        nowy_wiersz = []
```

```

        for i in range(liczba_wierszy):
            nowy_wiersz.append(macierz[i][j])
        wynik.append(nowy_wiersz)

    return wynik

def zeros(n,m):
    macierz = []

    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0)
        macierz.append(wiersz)

    return macierz

def macierz_odwrotna_Laplace(macierz): # punkt a
    n = len(macierz)
    d = wyznacznik_macierzy(macierz)

    if d == 0:
        raise ValueError("Macierz jest osobliwa, nie ma odwrotności")

    C = zeros(n, n)

    for i in range(n):
        for j in range(n):
            M = minor(macierz, i, j)
            C[i][j] = math.pow(-1, i+j) * wyznacznik_macierzy(M)

    Adj = transpozycja(C)

    wynik = zeros(n, n)
    for i in range(n):
        for j in range(n):
            wynik[i][j] = Adj[i][j] / d

    return wynik

def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    # macierz rozszerzona [A | I]
    rozszerzona_macierz = []

```

```

for i in range(n):
    wiersz = []

    # lewa strona: macierz A
    for j in range(n):
        wiersz.append(macierz[i][j])
        # wiersz.append(float(macierz[i][j]))

    # prawa strona: macierz jednostkowa I
    for j in range(n):
        if i == j:
            wiersz.append(1)
        else:
            wiersz.append(0)

    rozszerzona_macierz.append(wiersz)

# algorytm Gaussa-Jordana
for i in range(n):
    # jeśli na przekątnej jest 0, zamień wiersze
    if rozszerzona_macierz[i][i] == 0:
        znaleziono = False
        for k in range(i+1, n):
            if rozszerzona_macierz[k][i] != 0:
                rozszerzona_macierz[i], rozszerzona_macierz[k] =
rozszerzona_macierz[k], rozszerzona_macierz[i]
                znaleziono = True
                break
        if not znaleziono:
            raise ValueError("Macierz nie ma odwrotności")

    # dzielenie całego wiersza przez element główny
    element_glowny = rozszerzona_macierz[i][i]
    for j in range(2 * n):
        rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny

    # zerowanie pozostałych elementów w tej kolumnie
    for k in range(n):
        if k != i:
            wspolczynnik = rozszerzona_macierz[k][i]
            for j in range(2 * n):
                rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
wspolczynnik * rozszerzona_macierz[i][j]

odwrotna = []
for i in range(n):
    wiersz = []
    for j in range(n, 2 * n):

```

```
wiersz.append(rozszerzona_macierz[i][j])
odwrotna.append(wiersz)

return odwrotna

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

wynik_Laplace = macierz_odwrotna_Laplace(macierz)
wynik_Gauss_Jordan = macierz_odwrotna_Gaussa_Jordana(macierz)

print("Macierz odwrotna Laplace:")
for wiersz in wynik_Laplace:
    print(wiersz)

print("Macierz odwrotna Gauss Jordan:")
for wiersz in wynik_Gauss_Jordan:
    print(wiersz)
```

Zadanie 4. Napisz funkcję, która wykona mnożenie dwóch macierzy.

```
def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

macierz1 = [
    [1, 2],
    [3, 4]
]

macierz2 = [
    [5, 6],
    [7, 8]
]

wynik = mnozenie_macierzy(macierz1, macierz2)

print("Wynik mnożenia:")
for wiersz in wynik:
    print(wiersz)
```

Zadanie 5. Korzystając z rozwiązań poprzednich zadań wykonaj następujące mnożenia macierzowe: $A \cdot A^{-1}$ oraz $A^{-1} \cdot A$ i porównaj ich wyniki.


```

def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    # macierz rozszerzona [A | I]
    rozszerzona_macierz = []

    for i in range(n):
        wiersz = []

        # lewa strona: macierz A
        for j in range(n):
            wiersz.append(macierz[i][j])
            # wiersz.append(float(macierz[i][j]))

        # prawa strona: macierz jednostkowa I
        for j in range(n):
            if i == j:
                wiersz.append(1)
            else:
                wiersz.append(0)

        rozszerzona_macierz.append(wiersz)

    # algorytm Gaussa-Jordana
    for i in range(n):
        # jeśli na przekątnej jest 0, zamień wiersze
        if rozszerzona_macierz[i][i] == 0:
            znaleziono = False
            for k in range(i+1, n):
                if rozszerzona_macierz[k][i] != 0:
                    rozszerzona_macierz[i], rozszerzona_macierz[k] =
rozszerzona_macierz[k], rozszerzona_macierz[i]
                    znaleziono = True
                    break
            if not znaleziono:
                raise ValueError("Macierz nie ma odwrotności")

        # dzielenie całego wiersza przez element główny
        element_glowny = rozszerzona_macierz[i][i]
        for j in range(2 * n):
            rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny

        # zerowanie pozostałych elementów w tej kolumnie
        for k in range(n):

```

```

        if k != i:
            wspolczynnik = rozszerzona_macierz[k][i]
            for j in range(2 * n):
                rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
wspolczynnik * rozszerzona_macierz[i][j]

    odwrotna = []
    for i in range(n):
        wiersz = []
        for j in range(n, 2 * n):
            wiersz.append(rozszerzona_macierz[i][j])
        odwrotna.append(wiersz)

    return odwrotna

def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

macierz = [
    [2, 4, 6],
    [0, 2, -1],
    [-3, 3, 3]
]

macierz_odwrotna = macierz_odwrotna_Gausa_Jordana(macierz)

wynik1 = mnozenie_macierzy(macierz, macierz_odwrotna)
wynik2 = mnozenie_macierzy(macierz_odwrotna, macierz)

print("Wynik mnożenia A * A^-1:")
for wiersz in wynik1:

```

```
print(wiersz)

print("Wynik mnożenia  $A^{-1} * A$ :")
for wiersz in wynik2:
    print(wiersz)
```